Trust Flow-Driven Trusted Federated Learning in Edge Computing: A Study on Key Technologies

Anonymous Author(s)

Abstract. Federated learning (FL) supports collaborative edge intelligence without moving raw data, but open edge participation exposes aggregation to untrusted execution, model poisoning, stealthy backdoors, and severe Non-IID drift. Existing robust aggregation methods often tighten thresholds to reject malicious updates at the cost of removing legitimate long-tail clients. This paper proposes a trust-flow-driven trusted FL framework that couples hardware-anchored admission, runtime trust evidence, dual-stream reputation evolution, and risk-gated layered aggregation. The method separates long-term utility from short-term attack risk through a historical performance stream and an instant risk stream, then uses layer-wise gating, clipping, and survivor weight renormalization to suppress poisoned updates while preserving useful heterogeneous contributions. Experiments on Flower/PyTorch with CIFAR-10, 20 admitted clients, six mixed attack types, and five repeated runs show 92.31% accuracy and 10.21% ASR under a 30% malicious setting, with permanent-ban FPR close to zero.

Keywords: Federated Learning; Edge Computing; Trusted Execution Environment; Risk Modeling; Layered Aggregation; Backdoor Defense

1 Introduction

FL trains a global model through parameter exchange while keeping data on edge devices [1,14]. This design is well suited to edge intelligence, yet its aggregation phase becomes fragile when client identities, execution integrity, and update semantics cannot be fully trusted. Practical edge networks also exhibit device heterogeneity and strongly skewed data distributions, so robust aggregation has to distinguish malicious deviations from legitimate long-tail updates. This distinction becomes difficult under compound attacks, where adversaries may submit poisoned parameters, inject targeted backdoors, or exploit cross-round camouflage.

Prior work addresses different parts of this problem. Geometric and statistical defenses such as Krum and Trimmed Mean remove outlying updates [4,5], while FLTrust and RaSA use clean references or adaptive secure aggregation [6,7]. Backdoor-specific methods further examine trigger behavior or collusion patterns [2,3,9,10]. These mechanisms remain exposed to two recurring tensions: static software checks cannot guarantee that local execution was genuine, and

aggressive filtering often mistakes Non-IID long-tail updates for attacks. TEE-based FL systems provide a hardware root of trust [11,12,13,8], but they usually do not by themselves solve semantic poisoning after a client has been admitted.

This paper proposes *Trust Flow*, a full-process trusted aggregation framework for edge FL. The main idea is to make trust evidence flow across admission, auditing, temporal state evolution, and aggregation rather than treating each round as an isolated filtering decision. The contributions are:

1. A TEE-anchored dynamic admission mechanism that combines attestation-style admission evidence with runtime behavioral entropy and Kalman filtering.
2. A dual-stream reputation model that decouples historical utility from instant attack risk, reducing the tension between malicious recall and permanent false bans.
3. A layer-wise risk-gated aggregation rule that applies differentiated admission thresholds, dynamic L2 clipping, and survivor weight re-normalization.
4. An end-to-end evaluation under Non-IID CIFAR-10 and mixed attacks, with additional stress tests and ablations.

2 Background and Related Work

Federated learning in edge computing. In the standard FL workflow, clients complete local training and submit parameter updates to the server, which aggregates them and redistributes the global model. This workflow keeps data local but makes the server heavily dependent on the authenticity and quality of uploaded updates. Edge computing further complicates this setting because clients differ in compute capability, communication stability, and data distribution. As the system becomes more open and layered, a larger attack surface appears around client admission, local execution, and update aggregation.

Poisoning and backdoor attacks. Attacks in FL mainly include untargeted poisoning and targeted backdoors. Untargeted poisoning, such as sign flipping and gradient scaling, aims to disrupt convergence. Targeted attacks, including label flipping, trigger-based backdoors, clean-label poisoning, and semantic backdoors, try to preserve main-task behavior while implanting targeted misclassification. Backdoor attacks are especially challenging because a malicious update can remain close to normal directions while shifting hidden features [2,3]. In a multi-round setting, attackers may also behave normally for several rounds to accumulate trust before injecting low-amplitude or intermittent malicious updates.

Active defenses and their limits. Robust aggregation methods typically rely on geometric distances, coordinate-wise statistics, clean anchors, or adaptive aggregation. Krum and Trimmed Mean reject or suppress outlying updates [4,5]; FLTrust uses a server-side clean reference direction [6]; RaSA adapts secure aggregation under edge-assisted settings [7]. Backdoor-oriented defenses, including FLPurifier and RoseAgg, improve protection against trigger or collusion patterns

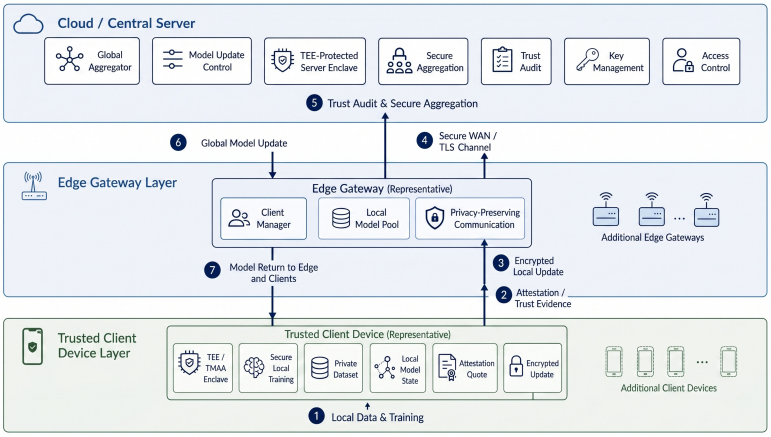


Fig. 1. TEE-anchored client-edge-cloud architecture.

[9,10]. These defenses remain sensitive to strong Non-IID distributions: a strict threshold raises benign false positives, whereas a loose threshold lets stealthy attacks pass.

TEE-assisted trusted training. Trusted execution environments (TEEs), including Intel SGX and ARM TrustZone, provide isolated execution and protected key material. Prior studies show their value in robust privacy-preserving FL, training integrity, attack mitigation, and trustworthy model training [11,12,13,8]. Nevertheless, TEE admission alone cannot decide whether a genuine training process produced semantically safe model updates. Trust Flow therefore combines hardware-rooted admission with cross-round risk modeling and layer-aware robust aggregation.

3 System Model and Threat Assumptions

The system contains a central server $\mathcal{S}$ and K edge clients $\mathcal{C} = \{c_1, \ldots, c_K\}$. In round t, admitted clients receive $W^{(t-1)}$, train locally, upload $\Delta W_k^{(t)}$, and the server updates $W^{(t)} = W^{(t-1)} + \mathrm{Agg}(\{\Delta W_k^{(t)}\})$. The global objective is

$$\min_W \mathcal{L}(W) = \sum_{k=1}^{K} p_k \mathcal{L}_k(W), \quad p_k = |\mathcal{D}_k| / \sum_j |\mathcal{D}_j|. \tag{1}$$

The edge setting follows a client-edge-cloud workflow. The client side provides local training and trust evidence, while the server audits updates, evolves trust states, and performs secure aggregation.

Threat model. The server is honest-but-curious and follows the aggregation protocol. Attackers may control admitted clients and perform label flipping, trigger backdoors, clean-label poisoning, semantic backdoors, sign flipping, and gradient scaling. This paper assumes an upper bound of $< 50\%$ malicious clients. The main experiments use 30% as a typical high-risk setting and additionally

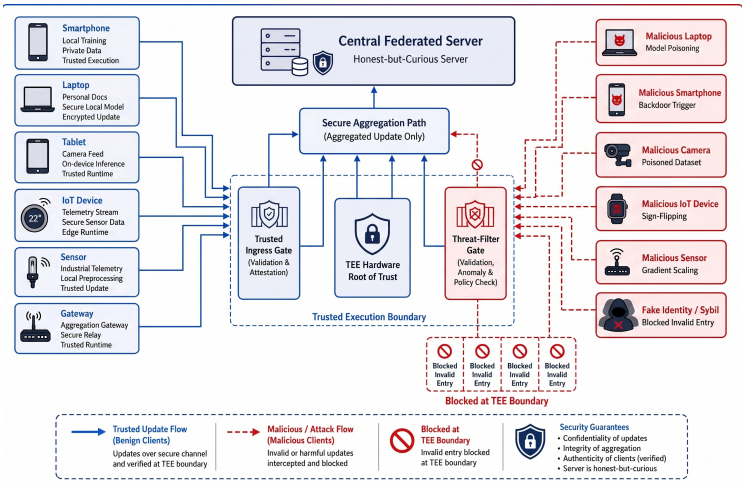


Fig. 2. Composite poisoning and penetration threat model.

report a 50% boundary stress test. TEEs are assumed to protect attestation-critical code and keys against a compromised host OS, excluding physical probing and advanced microarchitectural side-channel attacks.

Design goals. Trust Flow aims to: (i) maintain convergence under mixed poisoning and backdoors; (ii) keep permanent-ban false positive rate (FPR) low for legitimate long-tail clients; and (iii) avoid heavyweight cryptographic overhead while preserving edge deployability. Figures 1 and 2 summarize the physical workflow and the threat surface.

The central difficulty is that these goals are coupled. In a pure filtering strategy, strengthening the rejection rule improves malicious recall but also removes benign clients whose local distributions deviate from the majority. Conversely, relaxing the rule protects long-tail clients but allows low-amplitude poisoning to persist. Trust Flow treats this as a state-separation problem rather than a threshold-tuning problem: admission evidence, content quality, historical contribution, and instant risk remain visible as different signals until the final layer-wise aggregation decision.

4 Trust-Flow Framework

Trust Flow divides each FL round into four linked stages: admission and runtime trust assessment, reference-based content auditing, dual-stream state evolution, and risk-gated layered aggregation. Figure 3 shows the full defense loop.

At a higher level, the framework is a trust-state transfer pipeline. Phase one outputs $TrustScore$, which captures whether the client has a trustworthy basis for participation. Phase two outputs $ContentScore$, which measures whether the current update agrees with the clean reference and the peer group. Phase three deposits these signals into $HistPerf$ and $RiskEMA$, separating long-term contribution from short-term abnormality. Phase four fuses the states into

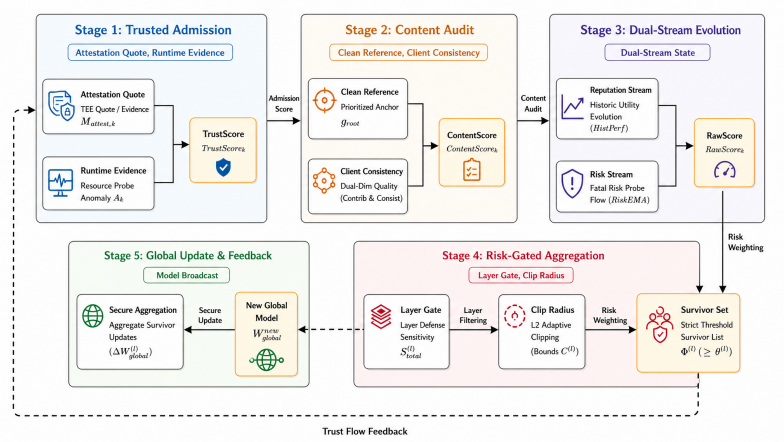


Fig. 3. Overview of the five-phase trust-flow defense loop.

RawScore and performs differentiated layer-wise aggregation. This design avoids treating a low single-round score as immediate evidence of maliciousness, which is important for long-tail clients under strong Non-IID data.

Operationally, each round follows the same decision path. The server first removes clients that fail the admission proof, since their updates cannot be tied to a protected execution path. It then scores the remaining updates against the clean reference and the trusted peer group, but still treats this score as provisional. The provisional evidence is accumulated into slow utility and fast risk states, and only then translated into layer-level aggregation privileges. This ordering is deliberate: early stages ask whether the evidence is credible, middle stages ask whether the current update is useful or risky, and the final stage asks where in the network the update should be allowed to contribute. The framework therefore avoids a single hard gate at the beginning of the round, which would be brittle under Non-IID data, while still enforcing hard isolation when risk evidence persists.

4.1 Admission and Content Auditing

Each admitted client must pass a TEE-style admission check. Let $M_{\text{attest},k} \in \{0, 1\}$ denote whether the attestation report is valid. During initialization, the client invokes a device-unique key and generates a remote attestation quote that includes integrity measurements such as PCR chains, code segment signatures, and runtime configuration. Only clients passing this check receive communication privileges. During local training, the trusted management agent monitors behavioral sequences $\mathcal{X}_k = \{\Delta\text{grad}, \Delta\text{loss}, \text{instr\_dist}\}$. Their entropy, $H(\mathcal{X}_k) = -\sum_i p(x_i) \log p(x_i)$, is mapped to a measurement uncertainty $R_k^{(t)} = \exp(\lambda H(\mathcal{X}_k^{(t)}))$. Intuitively, a client whose local training suddenly becomes high-entropy is not necessarily malicious, but its current evidence should be trusted less than evidence produced under stable execution.

Following the information-theoretic filtering view [15] and multidimensional trust evaluation [16], the trust estimate is updated as

$$\hat{T}_k^{(t)} = \hat{T}_k^{(t-1)} + K_k^{(t)}(Z_k^{(t)} - \hat{T}_k^{(t-1)}), \quad TrustScore_k = \hat{T}_k^{(t)}(1 + P_k^{(t)})^{-1}. \quad (2)$$

High-entropy runtime behavior therefore reduces the Kalman gain and prevents abrupt anomalous observations from being absorbed into the long-term trust estimate. The covariance $P_k^{(t)}$ serves as a confidence boundary: smaller covariance indicates more reliable trust estimation. Thus admission is not a one-time binary decision, but a continuously updated privilege factor that can be carried into later auditing and aggregation.

The server then constructs a clean guiding direction. If a small clean validation gradient is available, $g_{root} = g_{root\_clean}$; otherwise, it uses admitted clients with high trust and low previous risk, with reference weights $\omega_k^{ref} = TrustScore_k(1 - RiskEMA_k^{prev})^2$. This fallback matters in edge deployments where a server-side clean set may be small or absent: the reference should be conservative, but it should not be dominated by clients that were recently suspicious. For each update, content quality combines reference consistency and peer consistency:

$$S_{contrib,k} = \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, g_{root})), \quad (3)$$

$$S_{consist,k} = \frac{\sum_{j \neq k} TrustScore_j \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, \Delta W_j))}{\sum_{j \neq k} TrustScore_j + \varepsilon}, \quad (4)$$

$$ContentScore_k = \frac{(1 + \beta_{fusion}^2) S_{consist,k} S_{contrib,k}}{\beta_{fusion}^2 S_{consist,k} + S_{contrib,k} + \varepsilon}. \quad (5)$$

Here $\beta_{fusion} > 1$ increases the influence of the clean direction and helps resist collusive drift. The harmonic form is used deliberately: a client must be plausible both relative to the clean reference and relative to the trusted peer group. A high peer score alone is insufficient under collusion, while a high reference score alone may over-penalize legitimate Non-IID updates.

4.2 Dual-Stream State Evolution

Single reputation scores confuse low utility caused by data skew with high risk caused by attacks. Trust Flow therefore separates client state into a historical utility stream and an instant risk stream. The historical stream treats normalized content quality as soft evidence in a Beta reputation model. Good evidence increases α_k, weak evidence increases β_k, both with decay λ_h, and the expected historical utility is $HistPerf_k^{(t)} = \alpha_k^{(t)}/(\alpha_k^{(t)} + \beta_k^{(t)} + \varepsilon)$.

This channel is intentionally slow. A legitimate long-tail node may receive a low content score in a few rounds because its data distribution differs from the majority, not because it is attacking the model. A low $HistPerf$ therefore leads to reduced aggregation privilege or temporary soft isolation, rather than immediate permanent removal. The risk stream plays the

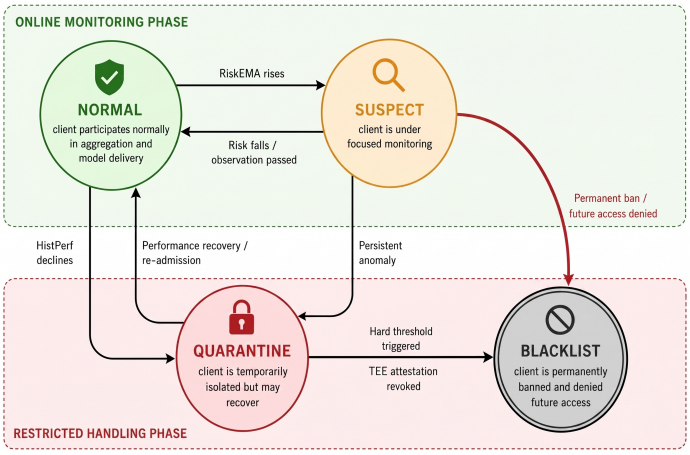


Fig. 4. Client state-transition automaton driven by historical utility and instant risk.

opposite role. It takes the maximum over several probes, including report anomalies, gradient direction/amplitude, clean-set loss increase, trigger activation drift, sign inconsistency, and peer disagreement, and then updates $RiskEMA_k^{(t)} = \beta_r RiskEMA_k^{(t-1)} + (1-\beta_r) Risk_k^{inst}$. A single strong attack signal is therefore not averaged away by benign historical behavior.

The separation also changes how borderline clients are treated. A client with weak content quality but no rising risk is kept in the system with reduced influence, giving later rounds a chance to confirm whether the low score was caused by Non-IID drift. A client with moderate historical utility but a sudden risk spike is handled in the opposite way: its current update is down-weighted or blocked before it can enter sensitive layers. This asymmetric response is central to the low permanent-ban FPR observed in the experiments, because the framework does not require the same threshold to serve as both a quality filter and an attack detector.

The two streams meet only when computing aggregation privilege:

$$RawScore_k = (TrustScore_k)^{\alpha}(ContentScore_k)^{\beta}(HistPerf_k^{(t-1)})^{\gamma} \times (1 - RiskEMA_k^{(t-1)})^{p}. \tag{6}$$

The multiplicative form makes the final privilege conservative: a client needs sufficient execution trust, current content quality, historical utility, and low recent risk. The resulting state machine distinguishes NORMAL, SUSPECT, QUARANTINE, and BLACKLIST states, as shown in Fig. 4. A sleeper-burst attacker may accumulate high $HistPerf$ during early benign rounds, but a later trigger injection activates the trigger or clean-probe channels and increases $RiskEMA$, blocking the attack even when historical utility is high. Conversely, a long-tail benign client with low current similarity is mainly handled through the slow utility stream and can recover after later positive contributions.

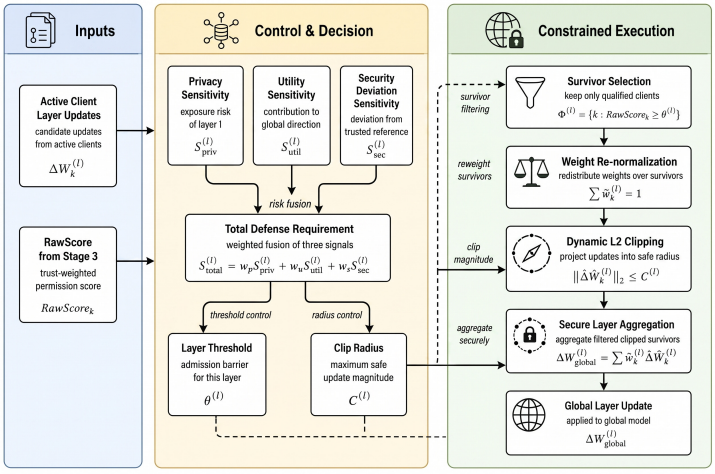


Fig. 5. Risk-gated layer-wise clipping and differentiated aggregation.

4.3 Risk-Gated Layered Aggregation

Backdoors often concentrate in deeper layers, whereas shallow feature layers may carry useful heterogeneous information. Trust Flow therefore assigns each layer l a sensitivity score: The score combines three signals: privacy exposure, reference-gradient utility, and security deviation. The privacy term decays with depth, the utility term measures the relative reference-gradient norm of the layer, and the security term measures how far trusted client updates drift from the reference direction. Their weighted sum $S_{total}^{(l)}$ controls both the admission threshold $\theta^{(l)} = \mu_{base} + \lambda_s S_{total}^{(l)}$ and the clipping radius $C^{(l)} = C_{base}/(S_{total}^{(l)} + \varepsilon_c)$. Thus a layer with stronger suspicious drift becomes harder to enter and more tightly clipped; a layer that remains close to the reference keeps more capacity for benign heterogeneity.

Survivors are $\Phi^{(l)} = \{k \in \mathcal{A} \mid RawScore_k \geq \theta^{(l)}\}$, and their weights are re-normalized:

$$\tilde{w}_k^{(l)} = \frac{RawScore_k}{\sum_{j \in \Phi^{(l)}} RawScore_j + \varepsilon}, \quad \widehat{\Delta W}_k^{(l)} = \frac{\Delta W_k^{(l)}}{\max(1, \|\Delta W_k^{(l)}\|_2 / C^{(l)})}. \tag{7}$$

The final update is $\Delta W_{global}^{(l)} = \sum_{k \in \Phi^{(l)}} \tilde{w}_k^{(l)} \widehat{\Delta W}_k^{(l)}$. Weight re-normalization prevents the removal of risky clients from shrinking the effective learning rate.

This layer-wise control is intended to preserve useful shallow representations while hardening layers that show stronger attack sensitivity. Consider a semantic backdoor that mainly shifts the final classifier or deep feature layers: a uniform global clipping rule must either be loose enough to preserve shallow features or tight enough to suppress the deep-layer attack, but it cannot do both well. Trust Flow allows those layers to follow different policies in the same round. The deep layer can reject low-privilege clients and reduce the update radius, while shallow

layers still use permissive thresholds when their reference consistency remains stable. This differentiated treatment is important because a uniform clipping radius can over-suppress benign feature extraction layers and slow convergence, especially after malicious clients have already been removed from the survivor set.

The re-normalization step is equally important for convergence. Without it, removing suspicious clients would reduce the sum of effective aggregation weights and create an implicit learning-rate decay. That decay is easy to overlook because it appears as a robustness decision, but it directly slows optimization under a fixed 30-round budget. Trust Flow therefore treats filtering and optimization as coupled operations: once the unsafe mass is removed, the remaining trustworthy mass is projected back to a valid aggregation simplex before clipping and layer assembly.

4.4 Analytical View

The dual-stream structure gives a simple way to interpret the FPR/ASR trade-off. Suppose the single-round content score of a legitimate long-tail client follows a perturbed distribution with expectation μ_{clean} and variance σ^2_{clean}. A rigid threshold can eliminate this client whenever its current score is below the threshold, even if the deviation comes from Non-IID data. After exponential smoothing, the steady-state variance of the historical utility estimate follows the standard low-pass form:

$$\mathrm{Var}(HistPerf_k^{(\infty)}) = \frac{1-\beta_h}{1+\beta_h}\sigma^2_{clean}. \tag{8}$$

As β_h approaches 1, short-term benign fluctuations are suppressed and the client is less likely to face sustained removal. In contrast, the instant risk stream responds to attack impulses. If a backdoor round produces an abnormal probe value, the max operator prevents that event from being averaged away before entering the EMA. Processing long-term contribution and short-term risk in separate spaces therefore reduces the single-score conflict: long-tail updates are smoothed rather than banned immediately, while abrupt malicious signals retain enough strength to trigger isolation.

4.5 Risk Probe Computation

The instant risk stream uses probes that are intentionally redundant rather than individually decisive. The gradient probe measures both direction deviation from g_{root} and abnormal update magnitude relative to the round statistics. This catches sign flipping and gradient scaling, but it is not sufficient for clean-label attacks whose directions may remain close to benign updates. The clean probe therefore applies a candidate update to a small server-side probe set and records the increase in cross-entropy loss. The trigger probe compares high-level activation histograms using KL divergence, $r^{(t)}_{trigger,k} = \mathrm{KL}(\mathcal{H}(A_{base}) \| \mathcal{H}(A_k))$,

which is more sensitive to semantic or clean-label backdoors that leave low-level gradients nearly unchanged.

These probes are fused through a maximum rule because their failure modes differ. A high-amplitude attacker is usually exposed by the gradient channel before it affects validation loss. A stealthy backdoor may instead appear as a modest clean-loss change but a clear activation shift. By preserving the strongest current warning, the risk stream makes each attack family visible to the state machine without requiring a single universal anomaly score.

The probes also provide useful diagnostic roles during auditing. When the gradient channel dominates, the server treats the client as an optimization-level threat and mainly suppresses its influence through clipping and layer thresholds. When the clean or trigger channel dominates, the update may look numerically ordinary but semantically unsafe, so the state machine pushes the client toward quarantine even if its historical utility is high. Peer disagreement is used as supporting evidence rather than a standalone reason for removal, because legitimate long-tail clients can disagree with the majority. This diagnostic separation makes the defense less dependent on a single score calibration and helps explain why the same mechanism can handle explicit parameter attacks, clean-label drift, and semantic backdoors in one pipeline.

5 Experiments

5.1 Setup

Experiments use Flower 1.5.0 and PyTorch on CIFAR-10 with a lightweight CNN. Simulations are deployed on an Inspur server with dual CPUs and five NVIDIA A10 GPUs. The software stack is Ubuntu 20.04.6 LTS and CUDA 12.8. To support reproducibility, complete logs, source code, and startup scripts are preserved. The 50,000 training images are split among 20 admitted clients. A shared 5,000-sample balanced pool supplies a common alignment basis; clients 6–11 follow Dirichlet $\alpha = 1.0$, and clients 12–19 use strong long-tail partitions with $\alpha = 0.1$.

The formal 30% attack setting uses six malicious clients: Client 0 performs label flipping with $\gamma = 0.5$; Client 1 injects a trigger backdoor into 20% of samples and targets class 0; Client 2 uses clean-label backdoor perturbations; Client 3 uses semantic backdoor bias; Client 4 applies sign flipping; and Client 5 amplifies updates by a factor of 100. Five Sybil nodes without valid certificates and three free-rider nodes with forged low workload are used to validate Phase-One gating. These eight nodes are intercepted before the statistical analysis, so the later metrics are computed over the 20 admitted clients. Each run uses 30 rounds and three local epochs. Aggregate Acc/ASR values are averaged over five seeds; state trajectories are shown from one representative seed.

Accuracy and ASR are reported as mean and standard deviation over five independent runs. Permanent-ban FPR is computed over benign clients that are irreversibly blacklisted, while temporary SUSPECT or QUARANTINE states

Table 1. Compact experimental configuration and main hyperparameters.

Category	Setting	Value
FL task	Dataset/model/admitted clients	CIFAR-10/light CNN/$K = 20$
Optimization	Local epochs, lr, momentum, rounds	3, 0.001, 0.9, 30
Data heterogeneity	Shared pool, Group A, Group B/C	5000, $\alpha = 1.0$, $\alpha = 0.1$
Trust streams	$\alpha_1, \alpha_2, \beta_{fusion}$; β_h, β_r	$0.6, 0.4, 2.0$; $0.9, 0.85$
Layer gating	fusion exponents; μ_{base}, λ_s	$3.0, 1.0, 0.5, 1.0$; $0.4, 1.2$

are treated as recoverable control actions. The single-run trajectory figures are taken from a fixed seed to make node-level transitions readable; aggregate tables use the repeated runs. All baselines are executed under the same node partitioning, attack proportion, optimizer configuration, communication rounds, and Non-IID split.

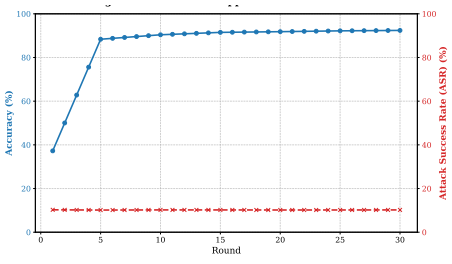
5.2 Overall Defense Performance

Figure 6 summarizes convergence, ASR, trust-state trajectories, and feature separation. Under 30% mixed attacks, Trust Flow converges to 92.31% accuracy and reduces ASR to 10.21%, close to the random-guess baseline for CIFAR-10. Client-state tracing shows that explicit poisoning and parameter attacks are blocked early, while stealthier clean-label drift is captured after persistent probe evidence. Legitimate long-tail nodes may enter SUSPECT or QUARANTINE temporarily, but their risk stream does not accumulate enough evidence for permanent banning.

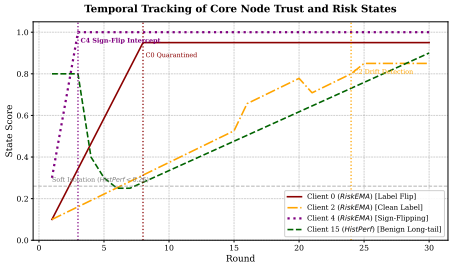
The node trajectories expose different decision paths. Label flipping and trigger backdoor clients show sustained high r_{grad} and r_{probe} values and enter the blacklist by round 8. Semantic backdoor behavior first causes soft isolation and then removal after persistent risk accumulation. The clean-label attacker is harder to detect because its local loss appears normal; it is identified after the drift-combination rule remains active for several rounds. Sign flipping sharply reduces the directional contribution score, whereas gradient scaling is caught by amplitude and layer-wise clipping. In contrast, the long-tail legitimate client may show low content quality early, but its risk stream does not keep rising, allowing it to recover through later $HistPerf$ updates.

The t-SNE visualization provides a complementary view. Before trust-flow intervention, malicious and legitimate long-tail clients can occupy overlapping regions because both may deviate from the majority direction. After dual-stream supervision, malicious clients form separable clusters associated with high-risk probes, whereas legitimate long-tail clients remain connected to the main aggregation region. This supports the design claim that historical utility and instant risk should not be collapsed into one score too early.

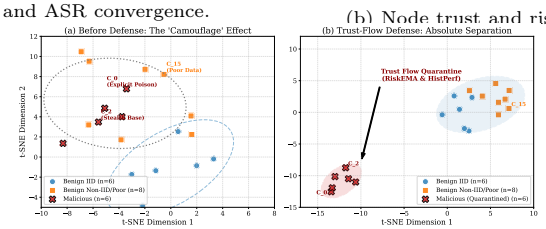
Compared with FedAvg, robust aggregation baselines reduce ASR but either lose substantial accuracy or remove many benign long-tail clients. Krum lowers



(a) Accuracy and ASR convergence.



(b) Node trust and risk trajectories.



(c) t-SNE separation before/after defense.

Fig. 6. Main defense behavior under mixed attacks.

Table 2. Performance comparison under 30% mixed attacks and boundary stress testing.

Method/Setting	Core mechanism	Acc.	ASR	Perm. FPR
FedAvg [14]	no defense	86.15 ± 1.20	92.34 ± 5.10	N/A
Krum [4]	Euclidean selection	64.20 ± 2.80	12.15 ± 0.90	64.20
Trimmed Mean [5]	coordinate trimming	71.35 ± 2.40	38.60 ± 4.20	N/A
FLTrust [6]	clean anchor weighting	81.25 ± 1.50	15.42 ± 1.10	21.40
Trust Flow	dual stream + layered gating	$\mathbf{92.31 \pm 0.09}$	$\mathbf{10.21 \pm 0.05}$	**0.00**
Boundary stress: 50% malicious	10/20 malicious	91.81 ± 0.10	10.46 ± 0.06	0.00

FPR denotes irreversible false bans of benign clients. Temporary SUSPECT/QUARANTINE states are recoverable and reported separately through trajectories.

ASR to 12.15% but its distance rule excludes many legitimate long-tail updates, leading to 64.20% accuracy. Trimmed Mean is more stable than Krum in some rounds, yet coordinate-wise truncation cannot suppress targeted backdoor behavior sufficiently. FLTrust benefits from a clean anchor but remains vulnerable when legitimate Non-IID updates deviate from the anchor. Trust Flow preserves benign heterogeneous contributions through historical smoothing, while the instant risk stream prevents high-reputation attackers from exploiting past behavior. The 50% result shows that the framework remains stable near the theoretical boundary, with 91.81% accuracy and 10.46% ASR.

5.3 Ablation, Sensitivity, and Overhead

Table 3 shows that a single-stream reputation model suffers from an FPR/TPR tradeoff: aggressive thresholds increase false bans, while conservative thresholds miss sleeper attacks. HistPerf-only protects benign nodes but lacks short-term

Table 3. Ablation and overhead summary.

Study	Variant / architecture	FPR or overhead	TPR / time
Reputation	Single-stream EMA, aggressive	18.25% FPR	95.12% TPR
Reputation	Single-stream EMA, conservative	2.10% FPR	48.33% TPR
Reputation	HistPerf-only	0.15% FPR	21.05% TPR
Reputation	**Dual-stream Trust Flow**	**0.05% FPR**	**98.50% TPR**
Overhead	Standard FedAvg	0 KB extra	2.10 s aggregation
Overhead	Trust Flow	∼15 KB TrustReport	4.85 s incl. audit

attack sensitivity. The dual-stream design reaches high recall while keeping permanent false bans low. The overhead is moderate: TrustReports add about 15 KB per upload, and server aggregation time increases from 2.10 s to 4.85 s per round.

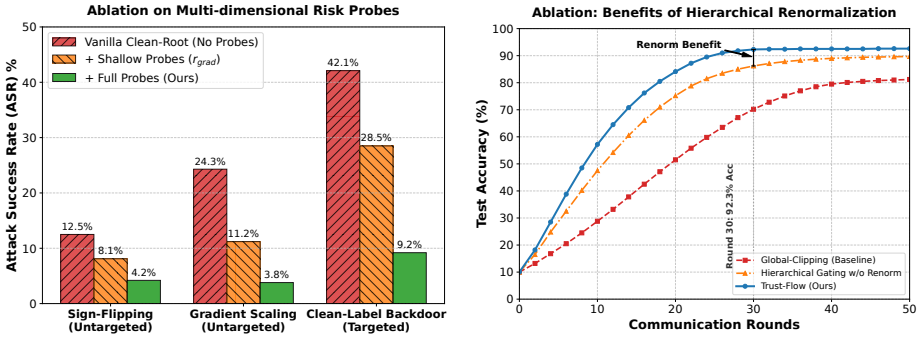
Figures 7 and 8 further decompose the effect of risk probes, weight renormalization, and data heterogeneity. Clean reference plus shallow probes handles sign flipping and gradient scaling, but clean-label backdoors remain difficult when gradient directions are collinear with benign updates. Adding cross-entropy and high-level activation probes reduces clean-label ASR below 9.2%. Re-normalization also matters: layer-wise gating without survivor weight correction creates an implicit learning-rate decay, while full Trust Flow converges within the 30-round budget. Under Dirichlet $\alpha = 0.1$, Krum and FLTrust degrade sharply, whereas Trust Flow remains near 92% accuracy.

The sensitivity curve confirms that traditional robust aggregation behaves well when data are close to IID, but deteriorates as α decreases. With $\alpha = 100$ or $\alpha = 1.0$, Krum and FLTrust remain close to the proposed method. In the long-tail setting $\alpha = 0.1$, however, their filtering rules increasingly treat legitimate drift as malicious deviation. Trust Flow is less sensitive to this shift because content evidence is not used alone; it is integrated with historical utility and instant risk. The fusion parameter β_{fusion} controls how strongly the clean reference dominates peer consistency. A smaller value retains more long-tail clients but raises ASR, while a larger value improves attack suppression but can over-penalize heterogeneous updates. The default $\beta_{fusion} = 2.0$ provides the best observed Acc/ASR balance.

The overhead analysis indicates that Trust Flow mainly shifts cost to the server. The edge side appends a TrustReport of roughly 15 KB and performs lightweight runtime monitoring. The server performs reference construction, pairwise consistency auditing, dual-stream updates, and layer-wise clipping, increasing aggregation time from 2.10 s to 4.85 s in the tested setup. Because wide-area edge FL rounds usually spend far more time on communication and synchronization than on aggregation arithmetic, this increase remains acceptable for the target scenario.

6 Discussion and Conclusion

Trust Flow integrates admission evidence, content auditing, temporal reputation, and layer-wise aggregation into a single trust-state pipeline. Its main bene-



(a) Multi-dimensional risk-probe ablation.

(b) Weight re-normalization effect.

Fig. 7. Component-level ablation of risk probes and re-normalization.

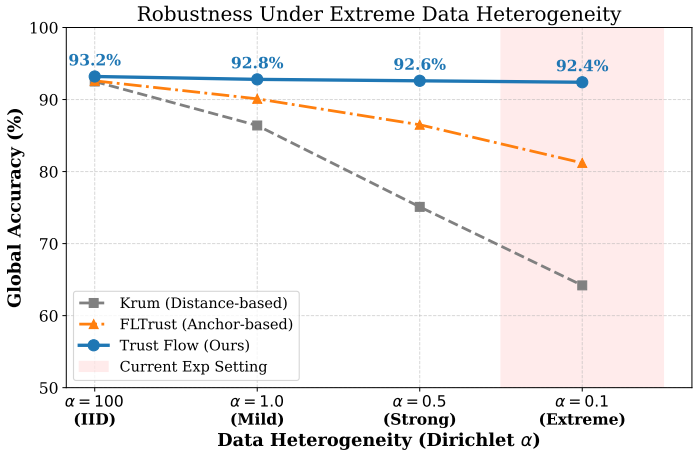


Fig. 8. Robustness stress test under varying data heterogeneity.

fit is not only stronger attack suppression, but a cleaner separation between two causes of low update similarity: benign long-tail data and malicious manipulation. This distinction allows the method to keep permanent-ban FPR near zero while maintaining high recall against mixed attacks.

Several limitations remain. Because publicly verifiable implementations are unavailable for several emerging TEE-FL methods, this study does not include them as direct baselines under the same experimental setting. The current probe design is also tailored to vision classification, especially CIFAR-10 backdoor behavior. Future work will release the source code and experimental scripts with the final manuscript to support reproduction, evaluate larger edge deployments, and extend the multidimensional risk probes to NLP and federated fine-tuning scenarios.

References

1. Kairouz, P., McMahan, H.B., Avent, B., et al.: Advances and open problems in federated learning. Foundations and Trends in Machine Learning 14(1–2), 1–210 (2021). https://doi.org/10.1561/2200000083
2. Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., Shmatikov, V.: How To Backdoor Federated Learning. In: Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, PMLR 108, pp. 2938–2948 (2020). https://proceedings.mlr.press/v108/bagdasaryan20a.html
3. Wang, B., Yao, Y., Shan, S., Li, H., Viswanath, B., Zheng, H., Zhao, B.Y.: Neural Cleanse: Identifying and mitigating backdoor attacks in neural networks. In: 2019 IEEE Symposium on Security and Privacy (SP), pp. 707–723 (2019). https://doi.org/10.1109/SP.2019.00031
4. Blanchard, P., El Mhamdi, E.M., Guerraoui, R., Stainer, J.: Machine learning with adversaries: Byzantine tolerant gradient descent. In: Advances in Neural Information Processing Systems 30 (2017). NeurIPS paper page
5. Yin, D., Chen, Y., Kannan, R., Bartlett, P.: Byzantine-robust distributed learning: Towards optimal statistical rates. In: Proceedings of the 35th International Conference on Machine Learning, PMLR 80, pp. 5650–5659 (2018). https://proceedings.mlr.press/v80/yin18a.html
6. Cao, X., Fang, M., Liu, J., Gong, N.Z.: FLTrust: Byzantine-robust federated learning via trust bootstrapping. In: Proceedings 2021 Network and Distributed System Security Symposium (NDSS) (2021). https://doi.org/10.14722/ndss.2021.24434
7. Wang, L., Huang, M., Zhang, Z., Li, M., Wang, J., Gai, K.: RaSA: Robust and adaptive secure aggregation for edge-assisted hierarchical federated learning. IEEE Transactions on Information Forensics and Security 20, 4280–4295 (2025). https://doi.org/10.1109/TIFS.2025.3559411
8. Lu, Z., Wang, L., Zhang, Z., Huang, M., Wang, J., Li, M.: TMT-FL: Enabling trustworthy model training of federated learning with malicious participants. IEEE Transactions on Dependable and Secure Computing 22(3), 2723–2740 (2025). https://doi.org/10.1109/TDSC.2024.3521377
9. Zhang, J., Zhu, C., Sun, X., Ge, C., Chen, B., Susilo, W., Yu, S.: FLPurifier: Backdoor defense in federated learning via decoupled contrastive training. IEEE Transactions on Information Forensics and Security 19, 4752–4766 (2024). https://doi.org/10.1109/TIFS.2024.3384846
10. Yang, H., Xi, W., Shen, Y., Wu, C., Zhao, J.: RoseAgg: Robust defense against targeted collusion attacks in federated learning. IEEE Transactions on Information Forensics and Security 19, 2951–2966 (2024). https://doi.org/10.1109/TIFS.2024.3352415
11. Zhang, X., Chen, Z., Chen, G., Feng, X., Shen, Q., Wu, Z.: RPPFL: Robust and privacy-preserving federated learning via trusted execution environments. In: ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing, pp. 1–5 (2025). https://doi.org/10.1109/ICASSP49660.2025.10889398
12. Chen, Y., Luo, F., Li, T., Xiang, T., Liu, Z., Li, J.: A training-integrity privacy-preserving federated learning scheme with trusted execution environment. Information Sciences 522, 69–79 (2020). https://doi.org/10.1016/j.ins.2020.02.037
13. Queyrut, S., Schiavoni, V., Felber, P.: Mitigating adversarial attacks in federated learning with trusted execution environments. In: 2023 IEEE 43rd International

Conference on Distributed Computing Systems (ICDCS), pp. 626–637 (2023). https://doi.org/10.1109/ICDCS57875.2023.00069
14. McMahan, B., Moore, E., Ramage, D., Hampson, S., Aguera y Arcas, B.: Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, PMLR 54, pp. 1273–1282 (2017). https://proceedings.mlr.press/v54/mcmahan17a.html
15. Chen, B., Liu, X., Zhao, H., Principe, J.C.: Maximum correntropy Kalman filter. Automatica 76, 70–77 (2017). https://doi.org/10.1016/j.automatica.2016.10.004
16. Liu, Y., He, Z., Liang, J., Li, Z., Deng, Q.: Multidimensional trust evaluation and task match based workers recruitment scheme for MCS. IEEE Transactions on Dependable and Secure Computing, 1–17 (2026). https://doi.org/10.1109/TDSC.2026.3652987